

Contents

5	Nonlinear equations	117
5.1	The problem	117
5.2	Slow convergence methods	118
5.2.1	Bisection Method	118
5.2.2	Fixed Point Iteration	120
5.3	Fast convergence methods	122
5.3.1	Newton's method	122
5.3.2	Secant method	126
5.4	Practical issues	128
5.5	Newton's method for systems of equations	129
5.5.1	Taylor's expansion	129
5.5.2	Systems of nonlinear equations	129
5.5.3	Newton's method applied to the system	130

Chapter 5

Nonlinear equations

5.1 The problem

In many practical applications, sometimes one needs to solve a nonlinear equation of the form

Problem 5.1.1 *Given $f : \mathbb{R} \rightarrow \mathbb{R}$, find $x \in \mathbb{R}$ such that*

$$f(x) = 0.$$

or finding the solution of systems of nonlinear equations

Problem 5.1.2 *Given $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$, find $\mathbf{x} \in \mathbb{R}^n$ such that*

$$F(\mathbf{x}) = \mathbf{0}.$$

Example 5.1.1 [Kepler's equation] Given $a, b \in \mathbb{R}$, find $x \in \mathbb{R}$ satisfying

$$x - a \sin x = b.$$

Example 5.1.2 Find $x \in \mathbb{R}$ satisfying

$$x - \tan x = 0.$$

Example 5.1.3 [System of equations] Find $(x, y, z) \in \mathbb{R}^3$ satisfying

$$\begin{aligned} xy &= z^2 + 1 \\ xyz + y^2 &= x^2 + 2 \\ e^x + z &= e^y + 3 \end{aligned}$$

Example 5.1.4 [Navigation equations]

The GPS receiver uses messages received from satellites to determine the satellite positions and time sent. The x , y , and z components of satellite position and the time sent are designated as $[x_i, y_i, z_i, t_i]$ where the subscript i denotes the satellite and has the value $1, 2, \dots, n$, where $n \geq 4$. When the time of message reception indicated by the on-board clock is $\tilde{t}_r$, the true reception time is $\tilde{t}_r + b$ where b is receiver's clock bias (i.e., clock delay). The message's transit time is $\tilde{t}_r + b - t_i$. Assuming the message traveled at the speed of light, c , the distance traveled is $(\tilde{t}_r + b - t_i)c$. Knowing the distance from receiver to satellite and the satellite's position implies that the receiver is on the surface of a sphere centered at the satellite's position with radius equal to this distance. Thus the receiver is at or near the intersection of the surfaces of the spheres if it receives signals from more than one satellite. In the ideal case of no errors, the receiver is at the intersection of the surfaces of the spheres.

The clock error or bias, b , is the amount that the receiver's clock is off. The receiver has four unknowns, the three components of GPS receiver position and the clock bias $[x, y, z, b]$. The equations of the sphere surfaces are given by:

$$(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2 = ([\tilde{t}_r + b - t_i]c)^2, \quad i = 1, 2, \dots, n$$

or in terms of pseudoranges, $p_i = (\tilde{t}_r - t_i)c$, as

$$p_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} - bc, \quad i = 1, 2, \dots, n.$$

These equations can be solved by algebraic or numerical methods.

5.2 Slow convergence methods

5.2.1 Bisection Method

The *bisection method* is guaranteed to converge for any continuous f :

- Step 1: Find an interval $[a_n, b_n]$ such that $f(a_n)$ and $f(b_n)$ have opposite signs
- Step 2: Put

$$x_n = \frac{a_n + b_n}{2} = \text{midpoint of } [a_n, b_n],$$

- Step 3: Define a new interval

$$[a_{n+1}, b_{n+1}] = \begin{cases} [a_n, x_n] & \text{if } f(a_n) \text{ and } f(x_n) \text{ have opposite signs,} \\ [x_n, b_n] & \text{if } f(b_n) \text{ and } f(x_n) \text{ have opposite signs,} \\ [x_n, x_n] & \text{if } f(x_n) = 0. \text{ Stop.} \end{cases}$$

- Step 4: Back to Step 2, replacing n with $n + 1$ until convergence.

The following MATLAB script demonstrates the use of the bisection method to solve $x - \cos(x) = 0$.

MATLAB script

```
a = 0;
b = 2;
f = @(x) x - cos(x);
flag = 0;
while (flag==0)
    xn = 0.5*(a+b);
    if (abs(f(xn)) < eps)
        fprintf('Zero of f is %16.14f \n',xn);
        flag = 1;
    elseif (f(a)*f(xn) < 0)
        b = xn;
    elseif (f(xn)*f(b) < 0)
        a = xn;
    end
end
end
```

An equivalent Python script is listed below.

Python script

```
import numpy as np
a = 0
b = 2
f = lambda x: x - np.cos(x)
flag = 0
eps = 1e-15
while (flag == 0):
    xn = 0.5*(a+b)
    if (np.abs( f(xn) ) < eps):
        print('Zero of f is %16.14f' % xn)
        flag = 1
    elif ( f(a)*f(xn) < 0):
        b = xn
    elif (f(xn)*f(b) < 0):
        a = xn
```

Convergence of the bisection method

The intermediate value theorem ensures there exists a solution x^* such that $a_n \leq$

$x^* \leq b_n$ for all n , and since

$$b_n - a_n = 2^{-n}(b_0 - a_0) \rightarrow 0 \text{ as } n \rightarrow \infty,$$

it follows that $x_n \rightarrow x^*$. However, the convergence is typically slow.

5.2.2 Fixed Point Iteration

Example 5.2.1 Suppose that we want to solve the equation

$$x = e^{-x}.$$

Graphically, the solution $x = x^*$ is the x -coordinate of the point where the curves $y = x$ and $y = e^{-x}$ intersect.

The plot shows that $0.5 < x^* < 0.6$. From a first guess $x_0 = 0.5$ we can try to generate more accurate approximations as follows:

$$x_1 = e^{-x_0} = 0.6065,$$

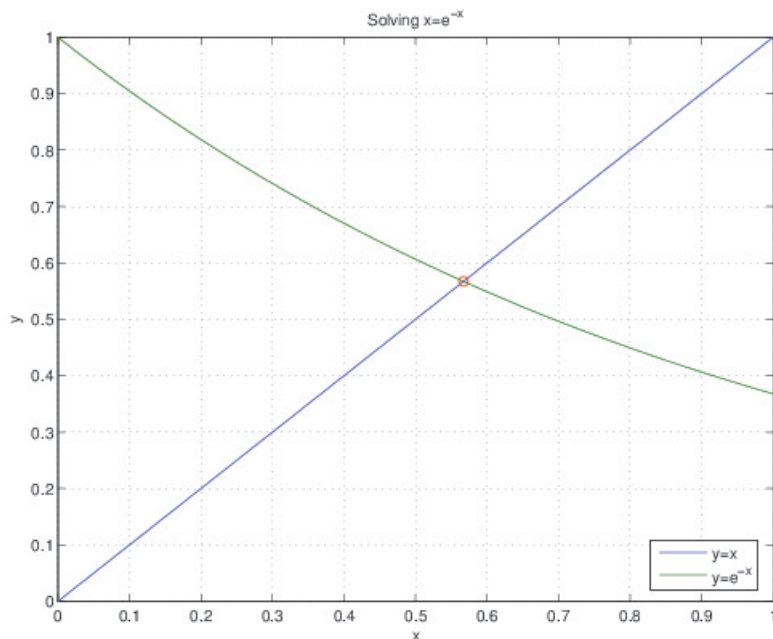
$$x_2 = e^{-x_1} = 0.5452,$$

$$x_3 = e^{-x_2} = 0.5797,$$

$$\vdots$$

$$x_{15} = e^{-x_{14}} = 0.5672,$$

$$x_{16} = e^{-x_{15}} = 0.5671.$$



A simple `for` loop may be used to perform the calculations and print a table:

MATLAB script

```
x = 0.5;
fprintf('    k          x_k\n\n')
for k = 1:15
    fprintf('%4d  %12.4f\n', k-1, x)
    x = exp(-x);
end
```

An equivalent Python script is listed below

Python script

```
import numpy as np
x = 0.5
print('    k          x_k \n')
for k in range(1,16):
    print('%4d  %12.4f  ' %(k-1,x))
    x = np.exp(-x)
```

The start of the output looks like

k	x_k
0	0.5000
1	0.6065
2	0.5452
3	0.5797
4	0.5601

In general, faced with an equation of the form

$$x = g(x),$$

we can try to compute a solution $x = x^*$ using *fixed-point iteration*:

- Choose a first guess x_0
- Compute $x_1, x_2, x_3, \dots$ using the recurrence

$$x_{n+1} = g(x_n) \quad \text{for } n = 0, 1, 2, \dots$$

The solution x^* is said to be a *fixed point* of the function g because $g(x^*) = x^*$.

To investigate the convergence (or otherwise) of x_n to x , we denote the error in the approximation $x_n \approx x^*$ by

$$e_n = x_n - x^*.$$

Observe that

$$e_{n+1} = x_{n+1} - x^* = g(x_n) - g(x^*) = g(x^* + e_n) - g(x^*).$$

Taylor expansion gives

$$g(x^* + e_n) = g(x^*) + g'(x^*)e_n + \frac{1}{2}g''(x^*)e_n^2 + \cdots,$$

so

$$e_{n+1} = g'(x^*)e_n + \frac{1}{2}g''(x^*)e_n^2 + \cdots.$$

Thus, if e_n is small,

$$e_{n+1} \approx g'(x^*)e_n.$$

Conclusion: We expect convergence if $|g'(x^*)| < 1$.

Note In practice, unless $g'(x^*)$ is quite small (say, less than 0.1 in magnitude) we are usually better off applying another method.

Example 5.2.2 Consider the equation

$$x = \frac{1}{\cosh x}.$$

The exact solution is $x^* = 0.76500995455073\dots$ and we see that

$$g(x) = (\cosh x)^{-1} \quad \text{and} \quad g'(x) = -(\cosh x)^{-2} \sinh x,$$

so $g'(x^*) = -0.4927\dots$. Starting a fixed point iteration from $x_0 = 0.75$ we find the following:

n	x_n	e_n	e_n/e_{n-1}
1	0.75000000	-1.5010e-02	
2	0.77238967	7.3797e-03	-0.4917
3	0.76137064	-3.6393e-03	-0.4932
4	0.76680210	1.7921e-03	-0.4924
5	0.76412679	-8.8316e-04	-0.4928
6	0.76544502	4.3507e-04	-0.4926
7	0.76479559	-2.1436e-04	-0.4927

5.3 Fast convergence methods

5.3.1 Newton's method

Any equation for x may be written in the form

$$f(x) = 0. \tag{5.3.1}$$

Solving the equation is equivalent to finding a zero of the function f , i.e., a number x^* such that $f(x^*) = 0$.

Given an approximate solution x_n , we observe that

$$f(x) = f(x_n) + f'(x_n)(x - x_n) + \frac{1}{2}f''(x_n)(x - x_n)^2 + \dots$$

By dropping the quadratic and higher-order terms, we arrive at an approximation to (5.3.1),

$$f(x_n) + f'(x_n)(x - x_n) = 0.$$

The solution $x = x_{n+1}$ of this *linearised equation* is

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Definition 1 (Newton's method)

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Geometrically, x_{n+1} is the x -intercept of the tangent line of the graph of f through the point $(x_n, f(x_n))$.

Example 5.3.1 We return to the equation

$$x = \frac{1}{\cosh x}.$$

The solution $x = x^*$ is a zero of the function $f(x) = x \cosh x - 1$. Taking $x_0 = 1$, Newton's method gives the following results.

n	x_n	$f(x_n)$
0	1.0000000000000000	5.4308e-01
1	0.8002117995531359	7.0382e-02
2	0.7658544692805376	1.6488e-03
3	0.7650104449251739	9.5681e-07
4	0.7650099545508975	3.2263e-13
5	0.7650099545507322	2.2204e-16
6	0.7650099545507321	-2.2204e-16

The following MATLAB script produces the previous table.

MATLAB script

```

N = 10;
x = 1;
fprintf('    k    x_k \n \n')
f = @(x) (x*cosh(x) - 1);
Df = @(x) (cosh(x) + x*sinh(x));
for k=1:N
    fprintf('%4d %18.16f  %3.4e \n',k-1,x,f(x))
    x = x - f(x)/Df(x);
end

```

An equivalent Python script is listed below.

Python script

```

import numpy as np
N = 10
x = 1
f = lambda x: (x*np.cosh(x)-1)
Df = lambda x: (np.cosh(x) + x*np.sinh(x))
print('    k        x_n                f(x_n)\n ')
for k in range(1,N+1):
    print('%4d    %18.16f  %3.4e  '% (k-1,x,f(x)))
    x = x - f(x)/Df(x)

```

We can view Newton's method for f as fixed point iteration for the function

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

Observe that

$$g'(x) = 1 - \frac{f'(x)f'(x) - f(x)f''(x)}{f'(x)^2} = \frac{f(x)f''(x)}{f'(x)^2},$$

so $g'(x^*) = 0$ provided $f'(x^*) \neq 0$, implying that the ratio e_{n+1}/e_n of successive errors tends to zero when the Newton iteration converges.

However, a bad choice of x_0 may result in $g(x_n)$ undefined.

Theorem 5.3.1 *Suppose that the Newton iterates x_n converge to a solution $x = x^*$ of the equation $f(x) = 0$. If $f'(x^*) \neq 0$, then the error $e_n = x_n - x^*$ satisfies*

$$\frac{e_{n+1}}{e_n^2} \rightarrow \frac{1}{2}g''(x^*) = \frac{f''(x^*)}{2f'(x^*)}.$$

Proof. By using the Taylor polynomial expansion around x_n ,

$$0 = f(x^*) = f(x_n) - f'(x_n)e_n + \frac{1}{2}f''(x_n)e_n^2 + O(e_n^3)$$

So

$$\begin{aligned} e_{n+1} &= e_n - \frac{f(x_n)}{f'(x_n)} \\ &= \frac{f'(x_n)e_n - f(x_n)}{f'(x_n)} \\ &= \frac{\frac{1}{2}f''(x_n)e_n^2 + O(e_n^3)}{f'(x_n)} \end{aligned}$$

Or equivalently

$$\frac{e_{n+1}}{e_n^2} = \frac{1}{2} \frac{f''(x_n)}{f'(x_n)} + \frac{O(e_n)}{f'(x_n)}.$$

Let x_n go to x^* we obtain the result. □

Example 5.3.2 Consider the function

$$f(x) = e^x + \log x, \quad x > 0.$$

Since $f(x) \rightarrow -\infty$ as $x \downarrow 0$ and $f(x) \rightarrow +\infty$ as $x \rightarrow \infty$, the graph of f must cut the positive x -axis at least once. Since $f'(x) = e^x + x^{-1} > 0$ for all $x > 0$, the function f is strictly increasing and so must cut the positive x -axis *exactly* once. Applying Newton's method with $x_0 = 1/2$, we observe the following.

n	x_n	e_n	e_{n+1}/e_n^2
0	0.5000000000000000	2.3013e-01	
1	0.2381071287046203	-3.1767e-02	-0.5999
2	0.2684966974527322	-1.3774e-03	-1.3650
3	0.2698717782430392	-2.3593e-06	-1.2435
4	0.2698741375665564	-6.8928e-12	-1.2383
5	0.2698741375734492	5.5511e-17	1168404.9785

Practical methods

- It can be shown that, provided $f'(x^*) \neq 0$, Newton's method converges whenever the initial guess x_0 is sufficiently close to the exact solution x^* .
- However, Newton's method can diverge if x_0 is badly chosen.
- Many applications call for an efficient method of solving $f(x) = 0$ that
 - is guaranteed to converge, and
 - needs values of $f(x)$ only, not of $f'(x)$.

5.3.2 Secant method

The *secant method* requires two (distinct) initial guesses x_0 and x_1 , and uses the iteration formula

$$x_{n+1} = x_n - \frac{f(x_n)}{f[x_{n-1}, x_n]},$$

where the divided difference is

$$f[x_{n-1}, x_n] = \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}.$$

- If x_n is close to x_{n-1} , then $f[x_{n-1}, x_n] \approx f'(x_n)$, so we can think of the secant method as an approximate Newton method.
- The name “secant method” arises from the fact that x_{n+1} is the x -intercept of the secant to the graph $y = f(x)$ through the points $(x_{n-1}, f(x_{n-1}))$ and $(x_n, f(x_n))$.
- The secant method does not require values of $f'(x)$, but its convergence is somewhat slower than Newton’s method.

Example 5.3.3 Consider once again the function

$$f(x) = e^x + \log x, \quad x > 0.$$

Applying the secant method with $x_0 = 0.5$ and $x_1 = 0.6$, we observe the following.

n	x_n	e_n	e_{n+1}/e_n^2
0	0.5000000000000000	2.3013e-01	
1	0.6000000000000000	3.3013e-01	
2	0.2313683391059020	-3.8506e-02	-0.5069
3	0.2808743090026827	1.1000e-02	-0.8654
4	0.2704226623870611	5.4852e-04	-1.2950
5	0.2698667958152603	-7.3418e-06	-1.2168
6	0.2698741425560123	4.9826e-09	-1.2372
7	0.2698741375734945	4.5297e-14	-1.2383
8	0.2698741375734492	5.5511e-17	245955.7785

Theorem 5.3.2 Suppose that the secant iterates x_n converge to a solution $x = x^*$ of the equation $f(x) = 0$. If $f'(x^*) \neq 0$, then the error $e_n = x_n - x^*$ satisfies

$$\frac{e_{n+1}}{e_n e_{n-1}} \rightarrow \frac{f''(x^*)}{2f'(x^*)}.$$

Proof Since $f(x^*) = 0$ we see that

$$\begin{aligned}
 e_{n+1} &= x_{n+1} - x^* = x_n - \frac{f(x_n)}{f[x_{n-1}, x_n]} - x^* = (x_n - x^*) - \frac{f(x_n) - f(x^*)}{f[x_{n-1}, x_n]} \\
 &= (x_n - x^*) - \frac{x_n - x^*}{f[x_{n-1}, x_n]} \frac{f(x_n) - f(x^*)}{x_n - x^*} \\
 &= (x_n - x^*) \left(1 - \frac{f[x^*, x_n]}{f[x_{n-1}, x_n]} \right) = e_n \frac{x_{n-1} - x^*}{f[x_{n-1}, x_n]} \frac{f[x_{n-1}, x_n] - f[x^*, x_n]}{x_{n-1} - x^*} \\
 &= e_n \frac{e_{n-1}}{f[x_{n-1}, x_n]} f[x_{n-1}, x_n, x^*]
 \end{aligned}$$

so

$$\frac{e_{n+1}}{e_n e_{n-1}} = \frac{f[x_{n-1}, x_n, x^*]}{f[x_{n-1}, x_n]} \rightarrow \frac{\frac{1}{2}f''(x^*)}{f'(x^*)}.$$

The MATLAB function **fzero** uses a combination of bisection, the secant method and *inverse quadratic interpolation*. We can use it to solve

$$e^x + \log x = 0, \quad x > 0,$$

as follows:

MATLAB script

```
>> f = @(x)exp(x)+log(x);
>> p = fzero(f, 1)
p = 0.26987413757345
```

Here, **fzero** looks for a solution near $x = 1$. Alternatively, we can supply an interval $[a, b]$ such that $f(a)$ and $f(b)$ have opposite signs.

MATLAB commands

```
>> f([0.1, 1.0])
ans = -1.19741417491840    2.71828182845905
>> p = fzero(f, [0.1, 1.0])
p = 0.26987413757345
```

An equivalent Python script is listed below.

Python script

```
import numpy as np
from scipy.optimize import root_scalar

# define the function
f = lambda x: np.exp(x) + np.log(x)
sol = root_scalar(f, method='toms748', bracket=[1e-3, 1])
print(sol.root)
```

5.4 Practical issues

To choose what method is appropriate for the problem, we need to consider the following.

- A graph is helpful in making decision of which method and initial guess to choose.
- If the function changes sign at a zero, the bisection method always finds the zero even though convergence can be slow.
- The fixed point iteration is simple, but is only efficient when $|g'(x)|$ is small.
- Newton's method and the secant method are very efficient and require a small number of iterations to find the zeros, but require good initial guesses. Use the graph.
- Newton's method requires one initial guess while the secant method requires two.
- The secant method is preferred over Newton's method if the evaluation of $f'(x)$ is difficult.

Stopping criteria

- Choose a tolerance τ . Use either
 - $|x_n - x_{n-1}| < \tau$ or
 - $|f(x_n)| < \tau$ or
 - $\frac{|x_n - x_{n-1}|}{|x_n|} < \tau$, if $x^n \neq 0$.
- It's a good practice to prescribe a maximum number of iterations.

Rate of convergence

Definition 2 Let $e_n = x^* - x_n$. The order of convergence is the largest ν such that

$$\lim_{n \rightarrow \infty} \frac{|e_{n+1}|}{|e_n|^\nu} = \beta < \infty.$$

- When $\nu = 1$ and $0 < \beta < 1$ we say that the method has *linear* or *first order convergence*.
 - When β is close to 1, the convergence is very slow.
 - The fixed point iteration has linear order convergence.

- If $\nu > 1$ the method is said to have *superlinear convergence order*. If $\nu = 2$ the method is said to have *quadratic convergence order*.
 - If $f'(x^*) \neq 0$, Newton's method has quadratic convergence order.
 - If $f'(x^*) \neq 0$, the secant method has superlinear convergence order with $\nu = (1 + \sqrt{5})/2 \approx 1.618$.

5.5 Newton's method for systems of equations

5.5.1 Taylor's expansion

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a smooth function. If $|h_1|, |h_2|, \dots, |h_n|$ are sufficiently small then

$$\begin{aligned} f(x_1 + h_1, \dots, x_n + h_n) \\ \approx f(x_1, \dots, x_n) + h_1 \frac{\partial f}{\partial x_1}(x_1, \dots, x_n) + \dots + h_n \frac{\partial f}{\partial x_n}(x_1, \dots, x_n). \end{aligned}$$

Example 5.5.1 If $f(x_1, x_2) = x_1^2 + x_2^2 + \ln x_1$, $(x_1, x_2) = (1, 2)$, $h_1 = 0.1$ and $h_2 = 0.2$ then

$$f(1.1, 2.2) \approx f(1, 2) + 0.1 \frac{\partial f}{\partial x_1}(1, 2) + 0.2 \frac{\partial f}{\partial x_2}(1, 2) = 6.1.$$

Exact value $f(1.1, 2.2) = 6.145310179804325$.

5.5.2 Systems of nonlinear equations

Consider the system

$$\begin{aligned} f_1(x_1, \dots, x_n) &= 0 \\ \vdots &= \vdots \\ f_n(x_1, \dots, x_n) &= 0 \end{aligned}$$

If $\mathbf{x}^* = (x_1^*, \dots, x_n^*)$ is a solution and $\mathbf{x}^{(k)} = (x_1^{(k)}, \dots, x_n^{(k)})$ is an approximate solution then

$$\begin{aligned} f_1(\mathbf{x}^*) &\approx f_1(\mathbf{x}^{(k)}) + (x_1^* - x_1^{(k)}) \frac{\partial f_1}{\partial x_1} + \dots + (x_n^* - x_n^{(k)}) \frac{\partial f_1}{\partial x_n} \\ \vdots &\approx \vdots \\ f_n(\mathbf{x}^*) &\approx f_n(\mathbf{x}^{(k)}) + (x_1^* - x_1^{(k)}) \frac{\partial f_n}{\partial x_1} + \dots + (x_n^* - x_n^{(k)}) \frac{\partial f_n}{\partial x_n} \end{aligned}$$

where the derivatives are evaluated at $\mathbf{x}^{(k)}$.

Let $\mathbf{f} = (f_1, \dots, f_n)$, and

$$J_{\mathbf{f}}(\mathbf{x}^{(k)}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}(\mathbf{x}^{(k)}) & \dots & \frac{\partial f_1}{\partial x_n}(\mathbf{x}^{(k)}) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1}(\mathbf{x}^{(k)}) & \dots & \frac{\partial f_n}{\partial x_n}(\mathbf{x}^{(k)}) \end{bmatrix}$$

Then

$$\mathbf{f}(\mathbf{x}^{(k)}) + J_{\mathbf{f}}(\mathbf{x}^{(k)})(\mathbf{x}^* - \mathbf{x}^{(k)}) \approx \mathbf{0},$$

Hence, assuming that the *Jacobian* $J_{\mathbf{f}}(\mathbf{x}^{(k)})$ is non-singular we have

$$\mathbf{x}^* \approx \mathbf{x}^{(k)} - J_{\mathbf{f}}(\mathbf{x}^{(k)})^{-1} \mathbf{f}(\mathbf{x}^{(k)}).$$

5.5.3 Newton's method applied to the system

Define

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - J_{\mathbf{f}}(\mathbf{x}^{(k)})^{-1} \mathbf{f}(\mathbf{x}^{(k)}).$$

Algorithm:

- Choose an initial guess $\mathbf{x}^{(1)} = (x_1^{(1)}, \dots, x_n^{(1)})$;
- For $k = 1, 2, 3, \dots$

– solve

$$J_{\mathbf{f}}(\mathbf{x}^{(k)}) \mathbf{d}^{(k)} = -\mathbf{f}(\mathbf{x}^{(k)}).$$

– define

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{d}^{(k)}$$

Note *The inverse of the Jacobian should never be explicitly calculated.*

Example 5.5.2 Find the solutions of

$$\begin{aligned} x_1^2 + x_2^2 - 8x_1 - 4x_2 + 11 &= 0 \\ x_1^2 + x_2^2 - 20x_1 + 75 &= 0 \end{aligned}$$

using the initial guess $(x_1^{(1)}, x_2^{(1)}) = (2, 4)$.

Let $\mathbf{f} = (f_1, f_2)$ where

$$\begin{aligned} f_1(x_1, x_2) &= x_1^2 + x_2^2 - 8x_1 - 4x_2 + 11 \\ f_2(x_1, x_2) &= x_1^2 + x_2^2 - 20x_1 + 75. \end{aligned}$$

On $k = 1$:

$$\mathbf{x}^{(1)} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}, \quad \mathbf{f}(\mathbf{x}^{(1)}) = \begin{bmatrix} -1 \\ 55 \end{bmatrix}, \quad J_{\mathbf{f}}(\mathbf{x}^{(1)}) = \begin{bmatrix} -4 & 4 \\ -16 & 8 \end{bmatrix}$$

On $k = 2$:

$$\begin{aligned} \mathbf{d}^{(1)} &= \begin{bmatrix} 7.125 \\ 7.375 \end{bmatrix}, & \mathbf{x}^{(2)} &= \mathbf{x}^{(1)} + \mathbf{d}^{(1)} = \begin{bmatrix} 9.125 \\ 11.375 \end{bmatrix}, \\ \mathbf{f}(\mathbf{x}^{(2)}) &= \begin{bmatrix} 105.156 \\ 105.156 \end{bmatrix}, & J(\mathbf{x}^{(2)}) &= \begin{bmatrix} 10.25 & 18.75 \\ -1.75 & 22.75 \end{bmatrix} \end{aligned}$$

On $k = 3$:

$$\begin{aligned} \mathbf{d}^{(2)} &= \begin{bmatrix} -1.5813 \\ -4.7439 \end{bmatrix}, & \mathbf{x}^{(3)} &= \mathbf{x}^{(2)} + \mathbf{d}^{(2)} = \begin{bmatrix} 7.5437 \\ 6.6311 \end{bmatrix}, \\ \mathbf{f}(\mathbf{x}^{(3)}) &= \begin{bmatrix} 25.0050 \\ 25.0050 \end{bmatrix}, & J(\mathbf{x}^{(3)}) &= \begin{bmatrix} 7.0874 & 9.2622 \\ -4.9126 & 13.2622 \end{bmatrix} \end{aligned}$$

k	$x_1^{(k)}$	$x_2^{(k)}$	$f_1(x_1^{(k)}, x_2^{(k)})$	$f_2(x_1^{(k)}, x_2^{(k)})$
1	2.000	4.000	-1.000e+00	5.500e+01
2	9.125	11.375	1.052e+02	1.052e+02
3	7.544	6.631	2.501e+01	2.501e+01
4	6.827	4.480	5.141e+00	5.141e+00
5	6.576	3.729	6.268e-01	6.268e-01
6	6.536	3.608	1.630e-02	1.630e-02
7	6.535	3.605	1.226e-05	1.226e-05
8	6.535	3.605	6.963e-12	6.992e-12

A simple Matlab program that produces the above table is given below.

MATLAB script

```
% Define the system of equations
f1 = @(x) x(1)^2 + x(2)^2 - 8*x(1) - 4*x(2) + 11;
f2 = @(x) x(1)^2 + x(2)^2 - 20*x(1) + 75;

% Define the Jacobian matrix
J = @(x) [2*x(1)-8  2*x(2)-4;
          2*x(1)-20 2*x(2)];

% Choose initial guess
x = [2;4];

% Set the maximum number of iterations
Nmax = 8;

% Start Newton's iteration
for k=1:Nmax
    vf = [f1(x); f2(x)];
    fprintf ('%6.3f %6.3f %6.3f %6.3f \n', x(1), x(2), vf(1), vf(2))
    d = J(x)\(-vf);
    x = x + d;
end
```

An equivalent Python script is given below.

Python script

```
import numpy as np

# Define the system of equations
f1 = lambda x: x[0]**2 + x[1]**2 - 8*x[0] - 4*x[1] + 11
f2 = lambda x: x[0]**2 + x[1]**2 - 20*x[0] + 75

# Define the Jacobian matrix
J = lambda x : np.array([ [2*x[0]-8, 2*x[1]-4],
                          [2*x[0]-20, 2*x[1]] ])

# Choose the initial guess as a column vector
x = np.array([2, 4]).T

# Set the maximum number of iterations
Nmax = 8

# Start Newton's iteration
for k in range(1,Nmax+1):
    vf = np.array([f1(x),f2(x)]).T
    print(' %6.3f %6.3f %6.3f %6.3f ' % (x[0],x[1],vf[0],vf[1]))
    d = np.linalg.solve(J(x), -vf)
    x = x + d
```